

Expression

林致宇

Expression

- Expression
 - $1 + 6$
 - $+$: Operator (運算子)
 - 1 and 6: Operand (運算元)
- Expression can also become an operand
 - $1 + 6 + 8$
 - $(1 + 6) + 8$

Expression

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int i, j, k;
7      i = 5 + (5 + 3) * 2;
8      j = (5 + i) * 2 + 3 * i;
9      k = (i + j) * (j - i);
10
11     cout << i << ", " << j << ", " << k << "\n";
12 }
```

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Assignment Operator

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      int i, j, k;
7      i = j = (k = 3) + 5; // Right-to-left
8      cout << i << ", " << j << ", " << k << "\n";
9
10 }
```

16	a?b:c throw co_yield = += -= *= /= %= <<= >>= &= ^= =	Ternary conditional ^[note 2] throw operator yield-expression (C++20) Direct assignment (provided by default for C++ classes) Compound assignment by sum and difference Compound assignment by product, quotient, and remainder Compound assignment by bitwise left shift and right shift Compound assignment by bitwise AND, XOR, and OR	Right-to-left
	17	, Comma	Left-to-right

Comment: I prefer using multiple lines.

老師的建議：
使用多行是沒有關係的！

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Arithmetic Operators

Operator	Description
+	Additive operator
-	Subtraction operator
*	Multiplication operator
/	Division operator
%	Remainder operator

Arithmetic Operators

```
4  int main() {  
5  
6      int i = 5, j = 3;  
7  
8      cout << (i + j) << "\n";  
9      cout << (i - j) << "\n";  
10     cout << (i * j) << "\n";  
11     cout << (i / j) << "\n"; // int, double  
12     cout << (i % j) << "\n";  
13  
14     cout << (double)(i / j) << "\n";  
15     cout << i / (double)j << "\n";  
16     cout << (double) i / j << "\n";  
17 }
```

Practice

- (Practice 1) Given 12345, output 1, 2, 3, 4, 5 (Given one integer number, output 5 integer numbers.)

```
int number = 67395;
```

```
int n5, n4, n3, n2, n1;
```

```
n5 = number/10000;
```

```
n4 = (number%10000)/1000;
```

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Unary Operators

Operator	Description
-	Negative value
++	Increment operator; increments a value by 1
--	Decrement operator; decrements a value by 1

Unary Operators

```
4  int main() {  
5  
6      int i = 5, j = 7;  
7  
8      cout << i++ << ", " << ++i << "\n";  
9  
10     cout << --j << ", " << j-- << "\n";  
11  
12     cout << i + (-j) << "\n";  
13     cout << i * (-j) << "\n";  
14  
15 }
```

老師的建議：

如果沒有把握，讓 `i++` 單獨成為一行敘述是沒有關係的！

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Compound Assignment Operator

```
4  int main() {  
5  
6      int i = 3;  
7      i += 5;  
8      cout << i << "\n";  
9      i *= 2;  
10     cout << i << "\n";  
11     i -= 7;  
12     cout << i << "\n";  
13 }
```

老師的建議：
寫 $i = i + 5$; 是沒有關係的！

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Comparison Operators

Operator	Description
==	Equal to (boolean, numeric)
!=	Not equal to (boolean, numeric)
>	Greater than (numeric)
>=	Greater than or equal to (numeric)
<	Less than (numeric)
<=	Less than or equal to (numeric)

Comparison Operators

```
4  int main() {  
5  
6      int i = 5, j = 5, k = 3, l = 7;  
7  
8      cout << (i < j) << "\n";  
9      cout << (i <= j) << "\n";  
10     cout << boolalpha;  
11     cout << (i <= j) << "\n";  
12     cout << (i != j) << "\n";  
13     cout << ((i != j) == (k > l)) << "\n";  
14     return 0;  
15 }
```

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Complement Operator

```
4  int main() {  
5  
6      int i = 5, j = 5, k = 3, l = 7;  
7  
8      cout << !(i < j) << "\\n";  
9      cout << !(i <= j) << "\\n";  
10     cout << boolalpha;  
11     cout << !(i <= j) << "\\n";  
12     cout << !(i != j) << "\\n";  
13     cout << !((i != j) == (k > l)) << "\\n";  
14     return 0;  
15 }
```

老師的提醒：
之後會學到 Bitwise complement operator

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Logical Operators

- Truth Table

p	q	AND (&&)	OR ()
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

Logical Operators

Operator	Description
&&	AND (short-circuit)
	OR (short-circuit)
!	NOT

老師的建議：

1. 不要使用複雜的運算元
2. 不要在運算元使用

Assignment Operator

Logical Operators

```
4  int main() {  
5  
6      int i = 5, j = 5, k = 3, l = 7;  
7  
8      cout << boolalpha;  
9      cout << (i != j && (k += 2) < l) << "\n";  
10     cout << k << "\n";  
11 }
```

將 && 改成 || 的執行結果是？

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Conditional Operator

- `_ ? _ : _`

```
4  int main() {  
5  
6      int i = 5, j = 7;  
7  
8      i = (i < j)? i+3 : j;  
9      cout << i << "\n";  
10 }
```

老師的建議：
若是自己寫程式，使用
if/else 也是可以的

Operators

- Assignment Operator
- Arithmetic Operators
- Unary Operators
- Compound Assignment Operators
- Comparison Operators
- Complement Operator
- Logical Operators
- Conditional Operator
- Bitwise Operators

Bitwise Operators

$\sim x = -x - 1$		AND		OR	XOR
p	~p	q	&		^
1	0	1	1	1	0
1	0	0	0	1	1
0	1	1	0	1	1
0	1	0	0	0	0

>> : right shift

<< : left shift

Bitwise Operators

```
4  int main() {
5
6      int i = 3, j = 5;
7
8      cout << ~j << "\n";
9      cout << (i & j) << "\n";
10     cout << (i | j) << "\n";
11     cout << (i ^ j) << "\n";
12     cout << (i >> 1) << "\n";
13     cout << (i << 1) << "\n";
14 }
```

$\sim j = -j - 1 \Rightarrow -5 - 1 = -6$

Bitwise Complement

AND: $\begin{array}{r} 0011 \\ 0101 \\ \hline 0001 \end{array} \Rightarrow (0001)_2 = 1_{10}$

OR: $\begin{array}{r} 0011 \\ 0101 \\ \hline 0111 \end{array} \Rightarrow (0111)_2 = 7_{10}$

XOR: $\begin{array}{r} 0011 \\ 0101 \\ \hline 0110 \end{array} \Rightarrow (0110)_2 = 6$

$0011 \Rightarrow 0001, (0001)_2 = 1$
相當於 $\div 2$ 取整

$0011 \Rightarrow 0110, (0110)_2 = 6$
相當於 $\times 2$

The Application of Bitwise Operators

- Bluetooth / Wi-Fi / NFC / GPS

```
4  int main() {  
5  
6      const int BLUETOOTH = 1;  
7      const int WIFI = 2;  
8      const int NFC = 4;  
9      const int GPS = 8;  
10     int setting = WIFI | GPS;  
11  
12     setting |= NFC;  
13  
14     if ((setting & NFC) == NFC) {  
15         cout << "NFC On" << "\n";  
16     }  
17     if ((setting & GPS) == GPS) {  
18         cout << "GPS On (1)" << "\n";  
19     }  
20     setting &= ~GPS;  
21     if ((setting & GPS) == GPS) {  
22         cout << "GPS On (2)" << "\n";  
23     }  
24 }
```

Associativity & Operator Precedence

Associativity & Operator Precedence

- Associativity

- `int i = 8 / 2 / 2;`
- `int i = (8 / 2) / 2;`
- `int i = 8 / (2 / 2);`

- Operator Precedence

- `int i = 3 + 5 * 2;`
- `int i = j+++k;`

老師的建議：
自己寫程式時，善用小括弧，不要為難自己

Precedence	Operator	Description	Associativity
1	::	Scope resolution	Left-to-right
2	a++ a-- type() type{} a() a[] . ->	Suffix/postfix increment and decrement Functional cast Function call Subscript Member access	
3	++a --a +a -a ! ~ (type) *a &a sizeof co_await new new[] delete delete[]	Prefix increment and decrement Unary plus and minus Logical NOT and bitwise NOT C-style cast Indirection (dereference) Address-of Size-of ^[note 1] await-expression (C++20) Dynamic memory allocation Dynamic memory deallocation	Right-to-left
4	.* ->*	Pointer-to-member	Left-to-right
5	a*b a/b a%b	Multiplication, division, and remainder	
6	a+b a-b	Addition and subtraction	
7	<< >>	Bitwise left shift and right shift	
8	<=>	Three-way comparison operator (since C++20)	
9	< <= > >=	For relational operators < and ≤ respectively For relational operators > and ≥ respectively	
10	== !=	For relational operators = and ≠ respectively	
11	&	Bitwise AND	
12	^	Bitwise XOR (exclusive or)	
13		Bitwise OR (inclusive or)	
14	&&	Logical AND	
15		Logical OR	

Source: https://en.cppreference.com/w/cpp/language/operator_precedence

16	a?b:c throw co_yield = += -= *= /= %= <<= >>= &= ^= =	Ternary conditional ^[note 2] throw operator yield-expression (C++20) Direct assignment (provided by default for C++ classes) Compound assignment by sum and difference Compound assignment by product, quotient, and remainder Compound assignment by bitwise left shift and right shift Compound assignment by bitwise AND, XOR, and OR	Right-to-left
17	,	Comma	Left-to-right

Source: https://en.cppreference.com/w/cpp/language/operator_precedence

Parenthesis

```
4  int main() {  
5  
6      int i = (8 / 2) / 2;  
7      cout << i << "\n";  
8  
9      int j = 3, k = 5;  
10  
11     cout << (j+(++k)) << "\n";  
12     cout << j << ", " << k << "\n";  
13 }
```